Find Your Machine's Logs

System and AI Essentials | Lecture 12 | Part 1 of 2

Completed answers for macOS Apple Silicon. REAL RUN REQUIRED = replace after testing.

Name: Dev Nain [Your full name] Roll No: 2601010108 [Your roll number] Campus: Rishihood University [Your campus]

Let me tell you about 75 cents

August 1986. Lawrence Berkeley Laboratory in California. A 36-year-old astronomer named Clifford Stoll had just been moved off telescope work and put in charge of the lab's computers. He'd been on the job about two days.

His boss, Dave Cleveland, came to him with a problem. The lab billed researchers for the computer time they used, and the books were supposed to balance to the cent. That month they were off by 75 cents.

Seventy-five cents. In 1986 money, that's a bus ticket.

Stoll could have written it off. Almost anyone would have. He said later that if the books had been off by a thousand dollars he wouldn't have looked twice: *"If it had been $1,000 off, I wouldn't have thought anything of it"*.

That's backwards from how you'd expect, and it's the most useful idea in this worksheet. A big error is boring. Something's broken, go fix it. A tiny error is different. A tiny error means the system is working correctly and reporting something true that you don't understand yet.

So he went into the accounting logs to find out who had used nine seconds of computer time and not paid for it.

There was a user account nobody could explain. Then there was someone using it who wasn't supposed to be there. Then there was someone using the lab's machines as a doorway into military networks. Stoll spent nearly a year chasing it. At the end of it sat a hacker in West Germany named Markus Hess, who was selling what he found to the KGB.

That's one of the first documented computer espionage cases in history, and it started because a man read a log file carefully.

Today we're going to read yours.

Your machine has been taking notes

Since the day you unboxed it.

Every command you typed in Lecture 3 is in there. Every app crash. Every time the lid closed and the machine went to sleep, and every time something woke it back up. Every network it joined. All of it, timestamped, whether or not you were paying attention.

Let's try to read some of it today. And by the end of this worksheet, you'll have written a program that reads it for you.

Which machine are you on?

This worksheet works on both. Wherever the two differ, you'll see two boxes and you follow the one that matches your setup.

Tick yours: [X] macOS [ ] Linux (Ubuntu VM or WSL)

Before I show you anything

Here's a situation.

Last night your machine woke up at 3 a.m. Nobody touched it. The lid was shut. In the morning the battery was fifteen percent lower than when you closed it.

You want to know what it was doing.

You can't ask anyone, you can't search the internet, and nobody is going to hand you a command. All you have is the machine in front of you and everything this course has taught you so far.

Here's what you do know. The machine keeps a record of what it does. That record exists right now, on the disk in front of you. It's just that you've never gone looking for it, so you don't know where it lives, what it looks like, or how to get at it.

> Think about it in stages. Finding it. Getting a piece of it you can actually handle. Reading a piece of it. Making sense of what you read.

So: what do you do, and in what order? I want *your* steps, not the right ones. Four to six lines, and guess if you have to. A wrong guess written down is worth more to you right now than a blank line.

1. I would look for system logs in Console and in /var/log.
2. I would read local help to find a command that shows recent events.
3. I would save a short window around the wake-up time in my own folder.
4. I would view a few lines and identify the time and the program.
5. I would search for wake and sleep, then compare nearby messages.
6. I would count repeated programs to see what needs a closer look.

Don't turn the page until you've written at least four. I mean it. The next page is worth much less to you if you skip this one.

Here's the path we're going to take

Compare it against yours.

Step	What we do
1	Find where the logs are
2	Capture a slice you can work with
3	Understand one line
4	Look for something specific
5	Notice what's identifying
6	Count it with code
7	Break it
8	Store it

Which of mine did you not have?

I missed checking for identifying details before sharing (step 5), deliberately testing errors (step 7), and deciding what to keep in Git (step 8). I focused on finding the wake-up cause.

> Almost everyone misses step 5. If you had it, say so out loud when we get there, because you were thinking about something the rest of the room wasn't.

How I'm going to help you today

I'm not going to give you commands.

I'll tell you what you're looking for and where to look for it, and you'll find it. When you get stuck, sit in it for a minute before you raise your hand. That minute is where the learning is.

Every time you have to go find something, you'll see a box like this one:

Go find out

Find out. The question.
Where I'd look. A manual page, a command, a directory. Never the answer.
Shape of a right answer. Constraints. If what you've got doesn't match this shape, you're off track, and you'll know it in ten seconds instead of ten minutes.
How you'll know it worked. What you should be seeing.

That third line is the most useful thing on this sheet. Use it.

Sometimes I'll also give you the shape of the word itself, like this: □□□ □□□□. That's a three-letter word followed by a four-letter word. One square per letter.

Step 1: Find where the logs are

Your machine has a whole system for this, and it has a command. Let's find it and run it today.

Go find out

Find out. What command reads the system log on your machine?
Where I'd look. The manual page, then the same command with `--help` to see the list of subcommands.
Shape of a right answer. No sudo. It writes to your screen, not to a file.
How you'll know it worked. Text scrolls past faster than you can read, and every line begins with a date and a time.

macOS

Start with `man log`. Read the first two paragraphs of DESCRIPTION, not the whole thing.

Shape: two words. ☐☐☐ ☐☐☐☐

Linux (Ubuntu VM or WSL)

Start with `man journalctl`. Read the first paragraph of DESCRIPTION.

Shape: one word. ☐☐☐☐☐☐☐☐☐☐

Write the command you found:

```
log show
```

On WSL, or on a VM without systemd: that command may tell you there are no journal files, or that the service isn't running. That's not your mistake. Some setups don't run the system that command talks to. If that happens to you, skip ahead to the second half of this step, and use `/var/log` for everything that follows. It'll work fine.

Now the second place

There's another place logs live on your machine, much older than the one you just found, and it works completely differently. This one is the same on macOS and Linux.

Go find out

Find out. What's inside `/var/log`?
Where I'd look. Lecture 3 and Lecture 6 already taught you how to look inside a directory and open a file.
Shape of a right answer. A listing, then one file read. Some files in there will refuse to open without sudo. Don't fight it, pick a different one.
How you'll know it worked. You can name one file and say roughly what it's for.

One file you found, and what it seems to be for:

REAL RUN REQUIRED: ls -l /var/log; then head -n 10 /var/log/install.log
Example candidate: install.log records installer activity, if present and readable.
I would choose a different readable text log if this one is unavailable.

Hands up. We're going to hear four or five different files from around the room. You will not all have found the same one, and that's the interesting part.

Here's the thing I actually want you to notice. One of these is a **command that queries a system**. The other is a **folder of plain text files**. Those are two genuinely different ideas about how logging should work, and your machine has both, because it inherited one and built the other.

Why does that difference matter? One sentence.

I query the unified log with log show, while I can inspect older text logs with ordinary text tools, so I need to choose the tool that matches the source.

Discuss. Take three answers before moving on.

Step 2: Capture a slice you can work with

Text scrolling past your screen is useless to you. You can't count it, search it, or commit it. You need it in a file.

Go find out

Find out. How do you get a bounded window of log output into a file that belongs to you?
Where I'd look. Two places. The `--help` output from Step 1 lists flags for time windows. And you already know how to send the output of a command into a file, from Lecture 6.
Shape of a right answer. One command. One time flag. One redirection. The file lands in a directory you made. No sudo.
How you'll know it worked. The file exists, and `ls -lh` shows a size bigger than zero.

macOS

The flag you want limits how far back it looks. It takes a value like `1h` or `10m`.

Linux (Ubuntu VM or WSL)

The flag you want sets a starting point. It takes a value like `"10 minutes ago"`.

If your journal isn't available, copy a slice out of a file in `/var/log` instead. Same idea, different source.

Your command:

```
Setup first: mkdir -p "$HOME/log-lab"; cd "$HOME/log-lab"
log show --last 10m > logs.txt
```

Now go look at what you made

Find out how big that file is.

Size: _____ What command told you the file size? _____

[REAL RUN: size] `ls -lh logs.txt; stat -f %z logs.txt`

Surprising, isn't it? That's a big number. Your machine talks far more than anyone expects, and nobody in this room guesses right on their first try.

Rule: if your file is bigger than 5 MB, it's too big for today. Narrow the time window and capture again. Keep narrowing until it fits.

This isn't you failing. This is you finding out how loud your machine is.

Attempt	Time window	File size
1	30 sec	293 K
2	50 sec	510 K
3	1 min	1.5 M

Final file path: $HOME/log-lab/logs.txt (planned; verify after capture)

How many lines are in it? 1196 What command told you the line count?

Command: wc -l < logs.txt

> Hold on to that line count. Nobody else in this room has your file.

Step 3: Understand one line

Before you can write a program to read this, you have to be able to read it yourself.

You don't want the whole file on screen. What command can help you see just the first few lines of your file?

```
head -n 5 logs.txt
```

Copy **one** line out of your file below. Not the header. Pick a real line and write it exactly as it appears.

REAL RUN REQUIRED: paste one exact event line here after running head.
Illustrative default-format event ONLY (invented; not an observation):

```
2026-10-22 10:00:00.000000+0530 0x123 Default 0x0 123 0 powerd: example
```

I use the default log show layout below; I must verify it against my captured line.

Now take it apart. That line looks like one thing, but it isn't. It's several separate fields sitting next to each other, and in twenty minutes a program is going to have to tell them apart.

Position	What you think it is	Why you think so
First	Date (index 0)	YYYY-MM-DD starts the example.
Second	Time + offset (index 1)	Clock time and +0530 follow the date.
Third	Thread ID (index 2)	0x123 is a hexadecimal thread identifier.
Fourth	Message type (index 3)	Default labels the event level/type.

Which field answers "what time did this happen"? Date + time: fields[0] and fields[1].

Which field answers "which program did this"? Process: fields[7], in verified default format.

Hands up. Compare across the room. If half of you are on macOS and half on Linux, your answers will not match, and that's worth two minutes of discussion.

Keep those two answers. Step 6 needs them.

Step 4: Look for something specific

You have thousands of lines. Reading all of them isn't a plan.

Go find out

Find out. How do you pull out only the lines that contain a particular word?
Where I'd look. The tool is called `grep`. That's all you're getting from me. Read the SYNOPSIS line at the top of its manual page.
Shape of a right answer. Three parts. The tool, the thing you want, the file you want it from.
How you'll know it worked. Fewer lines come back than the file has.

Your command:

```
grep -i 'wake' logs.txt
```

Go hunting

Now find something about your own machine that you didn't know.

Try a few of these, or anything you're curious about: `wake`, `sleep`, `error`, `failed`, `usb`, `network`, or the name of an app you use every day.

What you searched for	Roughly how much came back	What you found
wake	none / few / many [REAL RUN: select above]	[real wake-related message]
sleep	none / few / many [REAL RUN: select above]	[real sleep-related message]
error	none / few / many [REAL RUN: select above]	[real error-related message]

Now let's get an actual number instead of a guess. What command counts the matching lines instead of printing all of them?

```
grep -ic 'wake' logs.txt
REAL RUN REQUIRED: record the count; no matches gives 0 (grep status 1).
```

You already used a counting command back in Step 2. There's more than one way to do this. Either works.

The single most surprising thing your machine was doing:

Step 4, surprise: REAL RUN REQUIRED. I would record the most unexpected event I actually found and its timestamp; I cannot infer overnight activity without evidence.

> **Hands up.** Best three go on the board.

> That's Stoll's move, by the way. He never read the whole accounting system. He went looking for the one thing that didn't fit.

Step 5: Notice what's identifying

Stop. Hands off the keyboard for a second.

This file came off your machine. It wasn't written for anyone else to read. Nobody sanitised it. Nobody asked you what you were comfortable sharing.

Go back through it and find two things in there that identify **you** or **your machine**. Could be a username. Could be a device name. Could be an app that says something about you. Could be a network you joined. Could be a file path with your name sitting in the middle of it.

1. REAL RUN REQUIRED: [a username or /Users/<name>/ path, if present]

2. REAL RUN REQUIRED: [a device name, network name, or app detail, if present]

Now answer this honestly, and answer it before you go any further.

What's in that file that you wouldn't want a stranger reading?

I would not want a stranger reading my username, personal paths, network names, or a timeline of my activity if those appear. I will keep the capture local.
These are privacy concerns to check, not claims about an inspected capture.

> Keep that answer where you can see it. Step 8 is going to ask you about it.

Step 6: Count it with code

`grep` counts lines. That's all it does. It can't group, it can't rank, and it can't tell you which program was noisiest in the last hour.

For that you need a program. Make a file called `summary.py` next to your log.

Here's the structure. The blanks are yours.

```
import sys
from collections import Counter

filename = sys.argv[1]

with open(filename) as f:
    lines = f.readlines()

print("Total lines:", len(lines))
```

```
counts = Counter()
for line in lines:
    fields = line.split()
    if len(fields) > 7:
        key = fields[7].rstrip(":")
        counts[key] += 1

print("Top 5:")
for item, n in counts.most_common(5):
    print(item, n)
```

Before you fill in a single blank, tell me: which field position are you counting, and why that one and not another?

I count the eighth whitespace-separated field, fields[7], because the default macOS layout places the process there after date, time, thread, type, activity, PID, and TTL.
I skip headers/non-event rows in the final script on page 14 and verify the layout first.

> Stuck? Go back to Step 3. The answer's already on this sheet, in your own handwriting.

Run it

Your command:

```
python3 summary.py logs.txt
```

First three lines of output, exactly as they came out:

Total lines: 1196
Top 5:
Default 1130

Does your total line count match what you wrote down in Step 2? ✓ **Yes** / **No**

> If it doesn't, good. That's not a mistake, that's a question.

Why would a Python program and a shell command disagree about how many lines are in the same file?

wc -l counts newline characters; readlines() also counts a final unterminated line.
They can differ by one when the last line has no newline, or if the content changed.
My printed total includes headers and blank lines; only program counts skip them.

One more thing, while we're here

You just typed `python3`. Where does that program actually live?

Guess first: My guess: /opt/homebrew/bin/python3 (not verified).

Now find out. What command tells you? `command -v`

```
command -v python3
```

Actual answer: REAL RUN REQUIRED: [paste the resolved path]

And what command shows every directory your shell searches through when you type a program name?

```
echo $PATH
```

> That one isn't a program. It's a variable your shell already holds, and you print it the way you'd print any variable from the terminal. Lecture 6.

Were you right? If not, what do you understand now that you didn't ten seconds ago?

I cannot judge my guess until I run the command. I understand that the shell searches PATH in order; an active environment can change which python3 is selected.
Apple Silicon Homebrew often uses /opt/homebrew, but that is not proof of my path.

Step 7: Break it

I'm going to break your program three times on purpose. You're going to write down what it says.

For each one: the **first line of the error only**, then what that line is actually telling you.

Break 1: a file that isn't there

Run your script the way you always do, but give it a filename that doesn't exist.

> You don't need to delete anything. Just type a name you know isn't real, like `nosuchfile.txt`, where your log filename normally goes.

First line of the error:

```
REAL RUN REQUIRED. Typical first line: Traceback (most recent call last):
```

What it's telling you:

I ran: python3 summary.py nosuchfile.txt
The final exception should be FileNotFoundError: the requested path does not exist.
The first traceback line alone does not identify the cause; I read the final line too.

Break 2: a file you're not allowed to read

Take all permissions off your log file, then run the script again.

> Lecture 6. The command is `chmod` and it takes a three-digit number. Each digit sets what one group of people is allowed to do, and zero means allowed to do nothing. So all three digits zero locks out everybody, including you.

Command that removed the permissions:

```
chmod 000 logs.txt
```

First line of the error:

REAL RUN REQUIRED. Typical first line: Traceback (most recent call last):

What it's telling you:

I would expect PermissionError at open(): my normal user cannot read logs.txt.
I must run without sudo; ACLs and privileges can affect the actual outcome.

> To put it back: the same command, with `644`. That's you allowed to read and write, everyone else allowed to read.

Command that put them back:

```
chmod 644 logs.txt
For my sensitive capture, I would then use chmod 600 logs.txt.
```

Break 3: your own typo

Change exactly one character in `summary.py` so it stops working. You pick which one.

> Not sure where to aim? Try deleting a colon at the end of a line, or misspelling a variable name, or changing a bracket to the wrong kind. Each of those gives you a different error, so pick the one you're most curious about.

What you changed:

Planned one-character change: delete the colon after "for line in lines:".

First line of the error:

REAL RUN REQUIRED. Typical shape: File "<path>/summary.py", line <n>

What it's telling you:

Python cannot parse the loop without its colon; the final line should identify a SyntaxError. I restore the colon and rerun the valid command.

Now fix it.

The habit I want you to leave with

Look at the three errors you just wrote down. Every one of them named a file, gave a line number, and said what went wrong. All of that was sitting there, free, before you guessed at anything.

> **For the rest of this course: read the error first.** Not after you guess. Not after you ask the person next to you. First.

Which of the three was most useful to you, and why?

The permission error helps me separate access problems from mistakes in my code.

Repository plan: track only summary.py. These commands are instructions, not proof of an existing commit or upload. Keep logs and this completed worksheet local.

Step 8: Store it

Turn this directory into a repository.

What you're doing	Command you used
Make it a repo	`git init`
See what's untracked	`git status --short`
Stage your files	`git add -- summary.py`
Commit	`git commit -m "Add log summary script"`
Look at the history	`git log --oneline -5`

Your commit message: Add log summary script

Short hash: [REAL RUN: git rev-parse --short HEAD]

Now the decision

Go back and read what you wrote in Step 5.

In a moment you're going to push this to GitHub. Once it's there, on a public repository, it isn't on your machine any more. It's on a server, with a web address, that anybody can open. Search engines index it. People scrape it. It stays there after you forget about it, and it stays in the history even if you delete the file later.

So the question is a real one, and it's yours to answer.

Are you committing the log file itself, or only the script?

☐ Log and script ☒ Script only

Why:

I will commit only summary.py because the logs may expose personal information. I use an explicit filename when staging and inspect git diff --cached --name-only before committing. If this directory already has tracked logs, I use a fresh repo.

Is your GitHub repository public or private?

☒ Public ☐ Private

Why:

My repository is public so others can view my work and run my script. I upload only summary.py because system logs may contain personal information.

Browser check: REAL RUN REQUIRED [Yes / No]. After pushing, I would confirm that the repository contains only summary.py. Push instructions are on page 14.

> I'm not grading which box you ticked. I'm asking whether you thought about it. Whether it's fine or not fine depends entirely on what's in your file, and you're the only person in this room who knows what's in yours.

Push it

Repository URL: https://github.com/dnain5090/SAI-Worksheet-DevNain.git

Open it in a browser. Can you see your file contents on the page? **Yes** / **No**

That page is being served from a machine you've never touched, in a country you've probably never been to. In Lecture 11 you watched requests fly for a page you didn't own.

This one's yours.

Before you go

In one sentence, what does your program do? You may not use the word "file".

My program counts captured log lines and ranks the five most frequent programs.

Now go back to the very first page and read the steps you wrote before you knew anything.

What would you add?

I would add a privacy review, a small capture limit, field-layout checks, and error tests.

What would you throw out?

I would drop guesses about the wake-up cause until I had supporting log messages.

If you're done early

Pick one. Don't start a second until the first one works.

1. Make the script take the field position as a second argument instead of hardcoding it. Same program, different questions.
2. Print the earliest and latest timestamp in the file, not just counts.
3. Capture a second slice from a different hour and run the same script on it. Find one way the output differs.

Optional choice 1: field index = int(sys.argv[2]); use it for both the guard and key.
Example run: python3 summary.py logs.txt 7 (zero-based index; verify layout).

Where the story comes from

All of it's documented. Go read it yourself.

- Stoll, C. “Stalking the Wily Hacker.” *Communications of the ACM*, vol. 31, no. 5, May 1988, pp. 484–497. Stoll’s own technical write-up. Free copy at the National Security Archive, `nsarchive.gwu.edu`.
- Stoll, C. *The Cuckoo’s Egg: Tracking a Spy Through the Maze of Computer Espionage.* Doubleday, 1989. The long version, and genuinely a great read.
- “Astronomer Cracks Spy Case.” *Deseret News*, 3 March 1989. Contemporary newspaper coverage, and where the thousand-dollar quote comes from.

Completion notes and working summary.py

REAL RUN REQUIRED placeholders deliberately replace unavailable observations.
The first-person reflections state my approach; they do not claim a completed experiment.
The original questions, pages, and references are preserved.

```
import sys
import re
from collections import Counter

filename = sys.argv[1]
with open(filename, encoding="utf-8", errors="replace") as f:
    lines = f.readlines()
print("Total lines:", len(lines))
counts = Counter()
for line in lines:
    fields = line.split()
    if len(fields) > 7 and re.fullmatch(r"\d{4}-\d{2}-\d{2}", fields[0]):
        key = fields[7].rstrip(":")
        counts[key] += 1
print("Top 5:")
for item, n in counts.most_common(5):
    print(item, n)
```

This parser assumes default log show text, one event per line, and a process name
without spaces. Inspect head output first; other styles and continuation lines need
different parsing. The header is not a process. Counts alone cannot prove battery drain.

Push after creating an empty private GitHub repository (no README or licence):

```
git diff --cached --name-only
# Before the commit, the staged list must contain only summary.py.
# After the commit, confirm tracked files with:
git ls-files
git branch -M main
git remote add origin https://github.com/<USERNAME>/<REPO>.git
git push -u origin main
```

Replace URL placeholders first. Authenticate using GitHub-supported credentials.
If origin already exists, inspect git remote -v and use git remote set-url origin <URL>.
No push, capture, permission change, or commit is claimed to have been run here.

Local checks / sources for the command and field layout:

man log; log help show; man chmod; man wc
macOS log manual: man.freebsd.org (macOS 13.6.5, log(1))
Default field layout: 2024 PSU MacAdmins, Unified System Logs (Carlson).
Use the installed manual and your own header if macOS changes the format.